AutoOptions Recommendation Chatbot

I. Background Information and Motivation

Recent geopolitical instability has triggered significant volatility in precious metals like GLD and SLV, yet most investors face a massive "information gap" when trying to turn high-level macro indicators into specific options trades. To solve this, AutoOptions is a multimodal, multi-agent financial RAG platform designed to bridge the divide between complex market data and actionable strategy. By utilizing a Medallion architecture, the system turns heterogeneous signals into clear volatility-arbitrage opportunities, lowering the barrier to entry for both retail and non-expert users.

Our agent creates a unified source of truth by synthesizing three distinct data tracks into one cohesive intelligence layer. It establishes the current market regime by tracking macro-economic indicators like FRED interest rates and inflation, monitors daily market dynamics such as implied volatility and commodity price action, and gathers micro-level intelligence from real-time options chains and SEC filings, specifically Form 4 and 8-K, to detect insider activity and material events.

Ultimately, the core objective of AutoOptions is to merge unstructured sentiment from news and filings with structured hard numbers like volatility and liquidity. By bridging this divide, the system delivers comprehensive, traceable investment reports based on specific user queries. Every recommendation is grounded in a clear data lineage, including clickable trace-back links to original SEC sources, ensuring the output is both professional and easy to act on.

II. Problem Statement & Logic

Options trading is intimidating due to extreme data fragmentation and tight time sensitivity. Traders are often forced to juggle multiple sources across different timeframes, struggling to connect "semantic signals" (e.g., Fed rate hike narrative or insider trading flags in SEC filings) with "numeric truths" like implied volatility. Traditional tools fail to bridge this gap because they cannot effectively quantify unstructured data into actionable signals. Furthermore, conventional systems rely on rigid structures that miss non-linear trading opportunities and suffer from a "context-switch" problem, where they fail to dynamically adjust the importance of specific tickers or shifting market regimes.

Technically, standard vector retrieval is also too weak for finance because it lacks the metadata-aware filtering needed for precise option numeric. However, since options mechanics, such as strikes, expiries, and Greeks, are standardized across both individual stocks and commodity ETFs, our system can finally unify these data types. The Agent solves the context-switching pain point by leveraging the deep reasoning of LLMs to provide dynamic routing. After the RAG retrieval is complete, the system intelligently adapts its logic based on the asset: for commodity ETFs like GLD or SLV, the agent prioritizes macro data like FRED rates and geopolitical factors; for equities like NVDA or AAPL, it automatically activates a fundamental analysis agent focused on SEC Form 4 filings and corporate news. This ensures the reasoning always matches the specific asset class without manual intervention.

III. Data Sources and Methodology

AutoOptions integrates five distinct data families to capture both quantitative and qualitative market signals.

A. Quantitative Data Integration

On the quantitative side, the system relies on three primary sources to anchor its analysis in deterministic facts. First, the agent utilizes the **FRED API** to gather macroeconomic indicators, such as interest rates and inflation data, ensuring the AI agents maintain an up-to-date understanding of the broader economic environment. Second, **Yahoo Finance** serves as our primary source for daily market data, providing equity indices and precious metals options chains to track implied volatility and pricing dynamics. Finally, to monitor global systemic stress, the agent ingests the academic **Geopolitical Risk (GPR) Index**, which the agent statistically enriches by calculating rolling averages and historical percentiles to provide deep longitudinal context for the AI agents.

B. Qualitative Data Integration

For unstructured, qualitative data, our pipeline focuses on synthesizing market narratives and material corporate events. To capture global financial intelligence, we scrape real-time news articles via **GDELT**. Our pipeline utilizes **Llama 3** to translate foreign texts, generate sentiment scores, and identify specific asset classes and entities likely to be impacted by emerging narratives.

Additionally, the system monitors potential insider trading and major corporate catalysts by ingesting **Form 4** and **Form 8-K** filings directly from **SEC EDGAR**. We apply differentiated processing logic based on the document type:

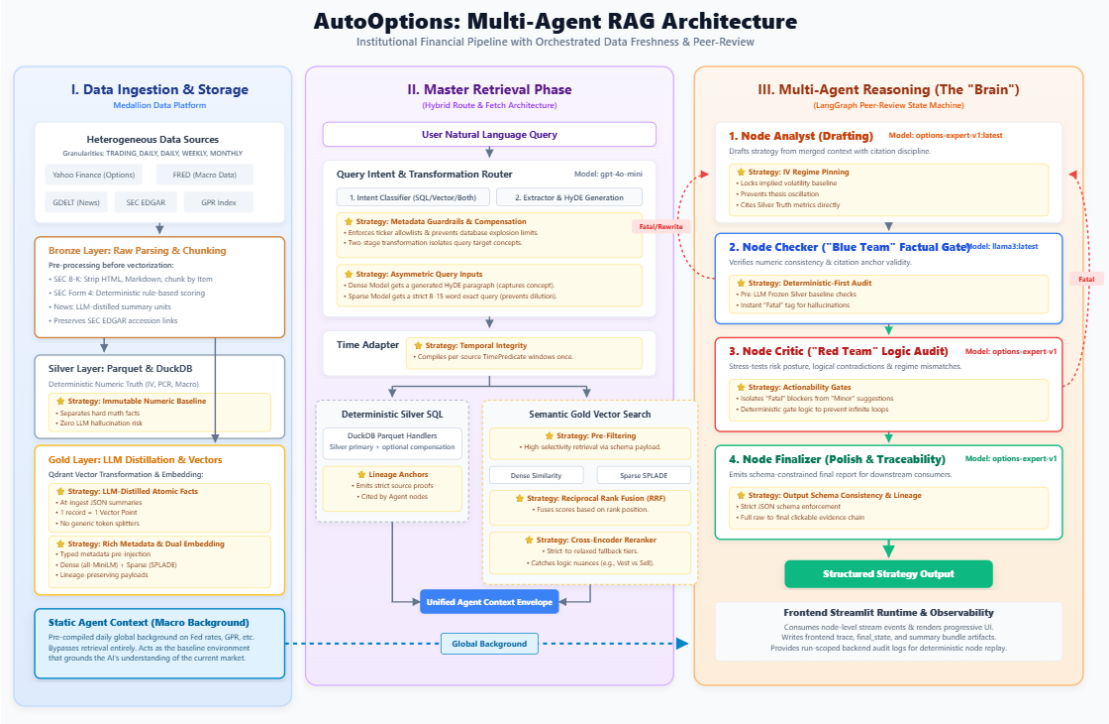
- **Form 4 (Insider Trades):** As these are provided in XML format, we parse the structured data into detailed transaction rows, tagging them with metadata such as reporting owner roles, transaction values, and precise timestamps for auditability.
- **Form 8-K (Material Events):** Due to their inherent complexity, consisting of dense legal language and messy financial tables, we implement a sophisticated preprocessing workflow. This includes **semantic chunking** by SEC "Item" type (e.g., Item 2.02 for earnings) and converting HTML tables into Markdown to preserve data logic. By stripping redundant SEC boilerplate and utilizing Llama 3 for targeted summarization, we ensure that only high-signal corporate disclosures are ingested into our retrieval layers.

C. Unified Processing Pipeline

All these diverse data sources go through a unified processing pipeline. They are first parsed and cleaned, then enriched with clear statistical or semantic signals, such as market direction or overall sentiment. Finally, they are stored consistently with a runtime anchor, ensuring that the AI can reliably and accurately query the data across different time horizons.

IV. System Architecture and Strategy

We utilized a Medallion Architecture to handle data lifecycle management and a LangGraph state machine for complex reasoning.



A. Data Pipeline (Medallion Layers)

The data pipeline for AutoOptions follows a strict Medallion architecture to ensure data integrity and complete traceability from raw ingestion to final analysis. By isolating data into distinct processing stages, the system guarantees that all investment recommendations are grounded in both verifiable historical evidence and real-time market signals.

The Bronze Layer

In the Bronze stage, the system ingests heterogeneous data formats (such as JSON, HTML, and XML) directly from primary sources like the FRED API for macroeconomic indicators, SEC EDGAR for corporate filings, and Yahoo Finance for market data. The primary objective of this layer is to preserve the data in its original form while extracting human-readable text and retaining the original source links. This structural preservation creates a comprehensive audit trail. Crucially, retaining these URLs allows the AI agents to include clickable, source-backed links in their final reports, establishing a verifiable ground truth that the end user can easily trace.

The Silver Layer

In this stage, the messy raw files are transformed into cleaned Parquet tables, which function as the platform's numeric system of record. This layer provides the deterministic truth required for quantitative financial analysis, housing precise metrics such as implied volatility, put-call ratios, and historical asset prices. By standardizing the data into a highly structured format, the system can execute high-performance SQL queries. These queries supply the AI agents with the exact mathematical figures they need to accurately evaluate market regimes and formulate trade structures without the risk of hallucination.

The Gold Layer

Finally, the data is refined into the Gold layer, where it is converted into a collection of high-signal

semantic artifacts. This layer stores information primarily in JSONL and Markdown formats, which are explicitly optimized for vector retrieval and Retrieval-Augmented Generation (RAG). Beyond structured facts, the Gold layer incorporates LLM-enriched qualitative signals, such as sentiment scores derived from financial news headlines and narrative summaries of geopolitical risks. This semantic layer serves as the primary reading material for the AI agents, enabling the system to seamlessly bridge the gap between hard quantitative numbers and complex market narratives.

B. Chunking Strategy

Chunking is one of the most consequential upstream decisions in our Retrieval-Augmented Generation (RAG) system. Because an incorrect strategy can severely degrade vector quality before any embeddings are even computed, the agent avoids using a generic, one-size-fits-all text splitter. Instead, since each data source has a fundamentally different information structure, the agent applies distinct, source-specific chunking policies designed to maximize the signal-to-noise ratio in the vector database.

SEC Filings: Text Synthesis and Semantic Chunking

For SEC filings, our overarching policy is to map one complete, self-contained filing record to exactly one Qdrant vector point without any arbitrary downstream splitting. However, the pre-vectorization strategies differ significantly between form types.

For Form 4 filings, which contain highly structured transaction data, we extract the raw XML fields and apply rule-based transformations to evaluate transaction polarity, pre-scheduled sale discounts, and executive roles. This synthesizes the machine-readable data into a single, high-density natural language sentence. Because each record acts as an atomic fact, splitting it further using standard token-based chunking would destroy its underlying meaning rather than preserve it.

Form 8-K filings require a more complex semantic chunking approach due to their heavy inclusion of legal boilerplate and noisy HTML. During ingestion, the system strips non-semantic elements, converts the text to Markdown, and systematically splits the document strictly by legal "Item" boundaries (e.g., Item 2.02 versus Item 8.01). We then feed these isolated semantic chunks into a llama3 to generate compact, structured JSON summaries.

Furthermore, we treat metadata as a first-class design component across all SEC chunks. Each vector payload merges the semantic text with deterministic metadata tags (e.g., ticker symbols, form types, and transaction dates) ensuring that retrieval remains operationally stable, precisely filterable, and fully traceable.

News and Narratives: LLM-Distilled Summary Units

For global news articles and Geopolitical Risk (GPR) narratives, we utilize an LLM-distilled chunking strategy. Rather than embedding the raw, noisy article text, we front-load the LLM processing (llama3:7B) during the initial ingestion phase. Our pipeline scrapes the raw data and immediately prompts an LLM to distill the content into a dense, coherent summary block equipped with a quantitative sentiment score. We then map one of these complete narrative

blocks directly to one vector point.

We intentionally avoid splitting these narrative articles sentence-by-sentence. Isolated sentences, such as "Gold rose," strip away the broader causal chain and consistently score weakly on semantic queries. Because our scraper-produced summaries already fall neatly within a compact 150 to 300-token window, they inherently act as the ideal chunk size. Applying additional token-based chunking to these texts would only create overlapping, incomplete sub-fragments, ultimately flooding our retrieval pool with low-precision noise.

C. Embedding Model Selection and Strategy

Because financial queries require both conceptual understanding and precise keyword matching, a single embedding model is insufficient. To address this, our system employs a tri-model architecture consisting of a dense encoder, a sparse encoder, and a cross-encoder reranker. All three models are specifically chosen to run efficiently on CPUs, strictly reserving our GPU resources for the primary LLM reasoning tasks.

Dense Encoder

For dense semantic retrieval, we utilize the `bge-base-en-v1.5` model. This model was selected because it achieves strong benchmark performance on financial information retrieval tasks, performing within a narrow margin of larger models while using 40% less memory. Operating in a 768-dimensional space, it natively produces L2-normalized unit vectors, which perfectly aligns with Qdrant's cosine similarity metric without requiring post-hoc score adjustments. This dense channel excels at capturing abstract market intents, such as a "flight to safety" or an "IV crush regime."

Sparse Encoder

To complement the dense semantic search, we deploy a SPLADE sparse encoder (`Splade_PP_en_v1`) via the `fastembed` library. Traditional keyword algorithms like BM25 struggle in the financial domain because they cannot handle abbreviation mismatches; however, SPLADE dynamically performs lexical expansion. For instance, a query containing "NVDA" seamlessly expands to include related terms like "NVIDIA" and "insider." This model preserves the high exact-match weight of critical financial tokens, such as ticker symbols and SEC form codes, ensuring that precise data is not lost during retrieval.

D. Retrieval Fusion and Filtering Strategy

When a user submits a query, the Retrieval Workflow begins with a smart router that classifies the underlying intent. The system dynamically routes the request into specific operating modes—such as SQL-only for strict numeric calculations, vector-only for narrative research, or a hybrid of both. To further enhance the search, the system occasionally uses Hypothetical Document Embeddings (HyDE) to generate a theoretical answer, which helps the vector database locate conceptually similar documents even if the exact keywords differ.

Asymmetric Input Design

The dense and sparse models are fed different variations of the user's query. The dense model receives a longer, LLM-generated Hypothetical Document Embedding (HyDE) paragraph, which

helps capture broad conceptual framing. Conversely, the sparse model is fed a highly factual, 8-to-15 word exact query(Sub-query). This deliberate separation prevents narrative filler from diluting the sparse model's keyword tracking while ensuring the dense model still activates deep semantic proximity.

Deterministic Pre-Filtering

Before Qdrant performs any vector math, the system applies deterministic payload pre-filtering. By leveraging extracted metadata, the vector database immediately filters out ineligible candidates based on ticker symbols, source types, SEC form codes, and sentiment scores. Furthermore, we enforce strict time barriers using an "OR" condition across multiple timestamp columns (`unified_timestamp` and `publish_timestamp`). This dual-key time filter maintains backward compatibility with legacy filings while guaranteeing that the AI only analyzes conceptually relevant and chronologically accurate context chunks.

Reciprocal Rank Fusion (RRF)

To unify the results from the dense and sparse channels, our Master Retriever utilizes Reciprocal Rank Fusion (RRF). We chose RRF over simple score averaging because dense cosine similarities and sparse dot products operate on entirely incompatible numerical scales. A basic average would allow one model to unfairly dominate the results based solely on mathematical artifacts. Instead, RRF converts the raw scores from both models into ranked lists and fuses them based on their rank positions. This rank-based approach correctly rewards documents that perform well across both retrieval methods while remaining immune to extreme outlier scores.

Cross-Encoder Reranker

Finally, we apply a cross-encoder reranker (`bge-reranker-v2-m3`) to the top candidates retrieved by the dense and sparse models. Unlike standard bi-encoders that rely on simple dot-product approximations, the cross-encoder analyzes the query and the document simultaneously in a single forward pass. This joint encoding allows it to catch subtle logical differences that are crucial in finance, effectively distinguishing between phrases that sound similar but mean opposite things (e.g., an insider "selling" versus simply "vesting" shares).

E. Multi-Agent Strategy

Instead of relying on a single, potentially unreliable LLM chatbot, AutoOptions utilizes a **multi-agent state machine** governed by LangGraph. This architecture prevents hallucinations by forcing the AI to peer-review its own work through a strict, iterative workflow.

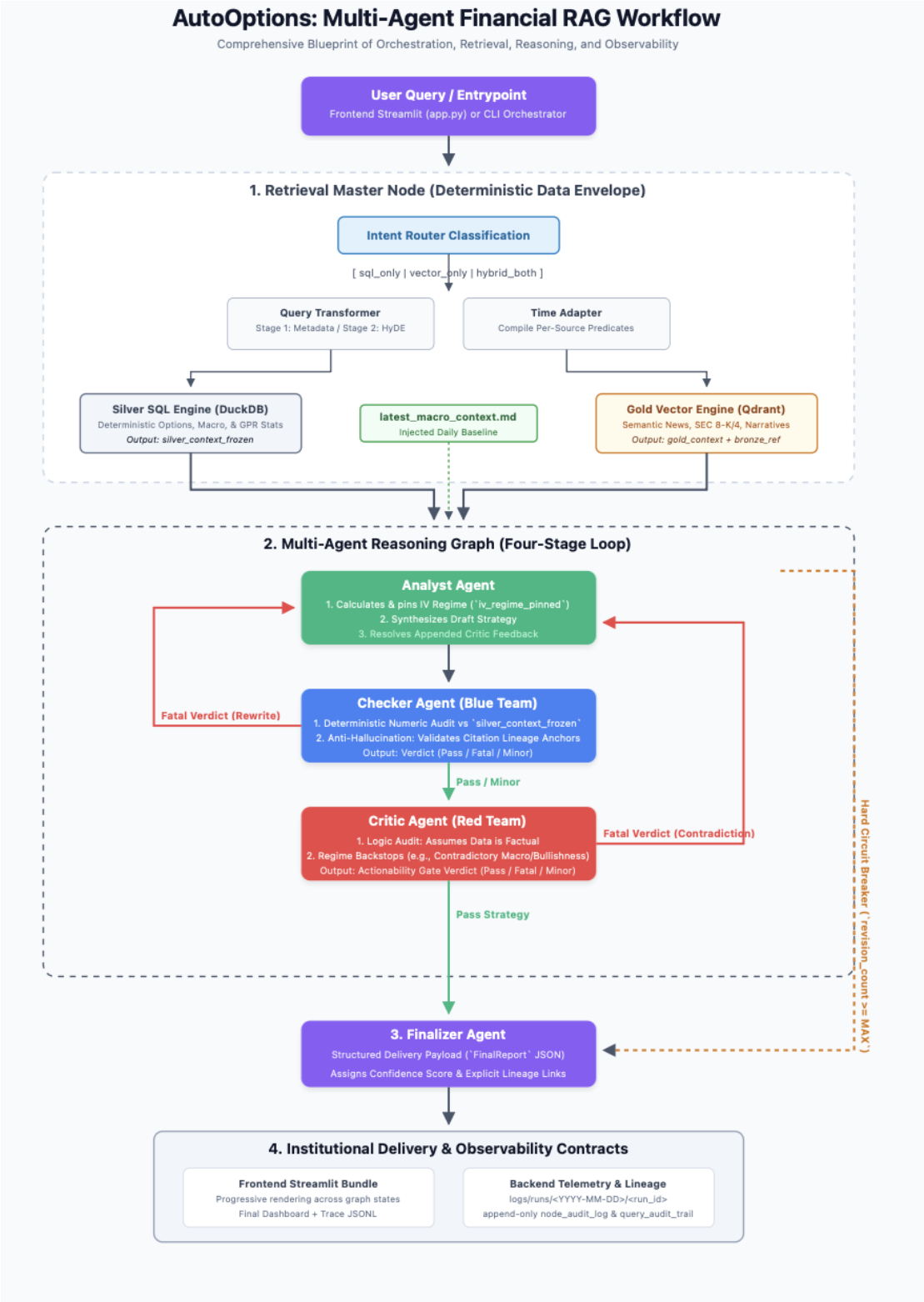
To maintain logical consistency across multiple rewrites, the **Analyst** employs a strategy called **"IV regime pinning."** By deterministically calculating and locking in the implied volatility regime during its first pass, the **Analyst** ensures that the core direction of the trade does not randomly shift if the report goes through multiple revision loops. Furthermore, to eliminate numeric drift, the **Checker** does not rely solely on AI intuition; it applies deterministic rule-based audits and regular expressions against a **frozen baseline of quantitative facts** before executing any LLM validation.

We also engineered strict feedback controls to manage the revision process. The **Critic** intentionally separates its feedback into **"fatal" issues**, which block the pipeline and require a complete rewrite, and **"minor" suggestions**, which are simply passed along for final polish. To prevent the system

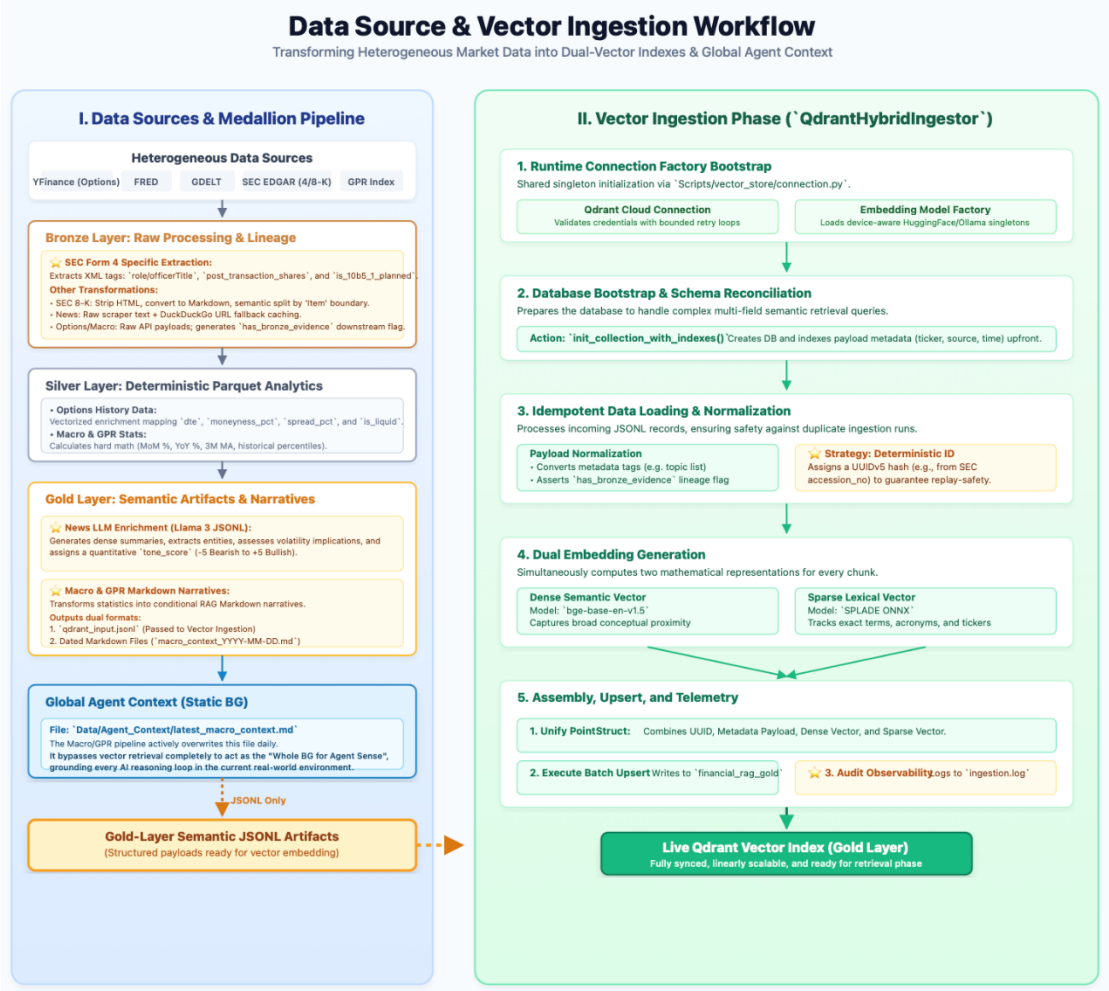
from getting stuck in an endless loop of false-fatal rejections, the **Critic** utilizes deterministic **actionability gates**.

Lastly, the **Finalizer** enforces rigorous **output schema consistency** so the final report never degrades into a disorganized text block. It ensures strict citation discipline by carrying forward complete **evidence traceability**, embedding explicit lineage links back to the original SEC filings or news articles, and ultimately assigning a **deterministic confidence score** based on the depth of the revisions.

V. Workflow


AutoOptions: Multi-Agent Financial RAG Workflow
Comprehensive Blueprint of Orchestration, Retrieval, Reasoning, and Observability
User Query / Entrypoint
Frontend Streamlit (app.py) or CLI Orchestrator
1. Retrieval Master Node (Deterministic Data Envelope)
Intent Router Classification
[sql_only | vector_only | hybrid_both]
Query Transformer
Stage 1: Metadata / Stage 2: HyDE
Time Adapter
Compile Per-Source Predicates
Silver SQL Engine (DuckDB)
Deterministic Options, Macro, & GPR Stats
Output: silver_context_frozen
latest_macro_context.md
Injected Daily Baseline
Gold Vector Engine (Qdrant)
Semantic News, SEC 8-K/4, Narratives
Output: gold_context + bronze_ref
2. Multi-Agent Reasoning Graph (Four-Stage Loop)
Analyst Agent
1. Calculates & pins IV Regime (`iv_regime_pinned`)
2. Synthesizes Draft Strategy
3. Resolves Appended Critic Feedback
Checker Agent (Blue Team)
1. Deterministic Numeric Audit vs `silver_context_frozen`
2. Anti-Hallucination: Validates Citation Lineage Anchors
Output: Verdict (Pass / Fatal / Minor)
Fatal Verdict (Rewrite)
Pass / Minor
Critic Agent (Red Team)
1. Logic Audit: Assumes Data is Factual
2. Regime Backstops (e.g., Contradictory Macro/Bullishness)
Output: Actionability Gate Verdict (Pass / Fatal / Minor)
Fatal Verdict (Contradiction)
Pass Strategy
Hard Circuit Breaker (`revision_count >= MAX`)
3. Finalizer Agent
Structured Delivery Payload (`FinalReport` JSON)
Assigns Confidence Score & Explicit Lineage Links
4. Institutional Delivery & Observability Contracts
Frontend Streamlit Bundle
Progressive rendering across graph states
Final Dashboard + Trace JSONL
Backend Telemetry & Lineage
logs/runs/<YYYY-MM-DD>/<run_id>
append-only node_audit_log & query_audit_trail

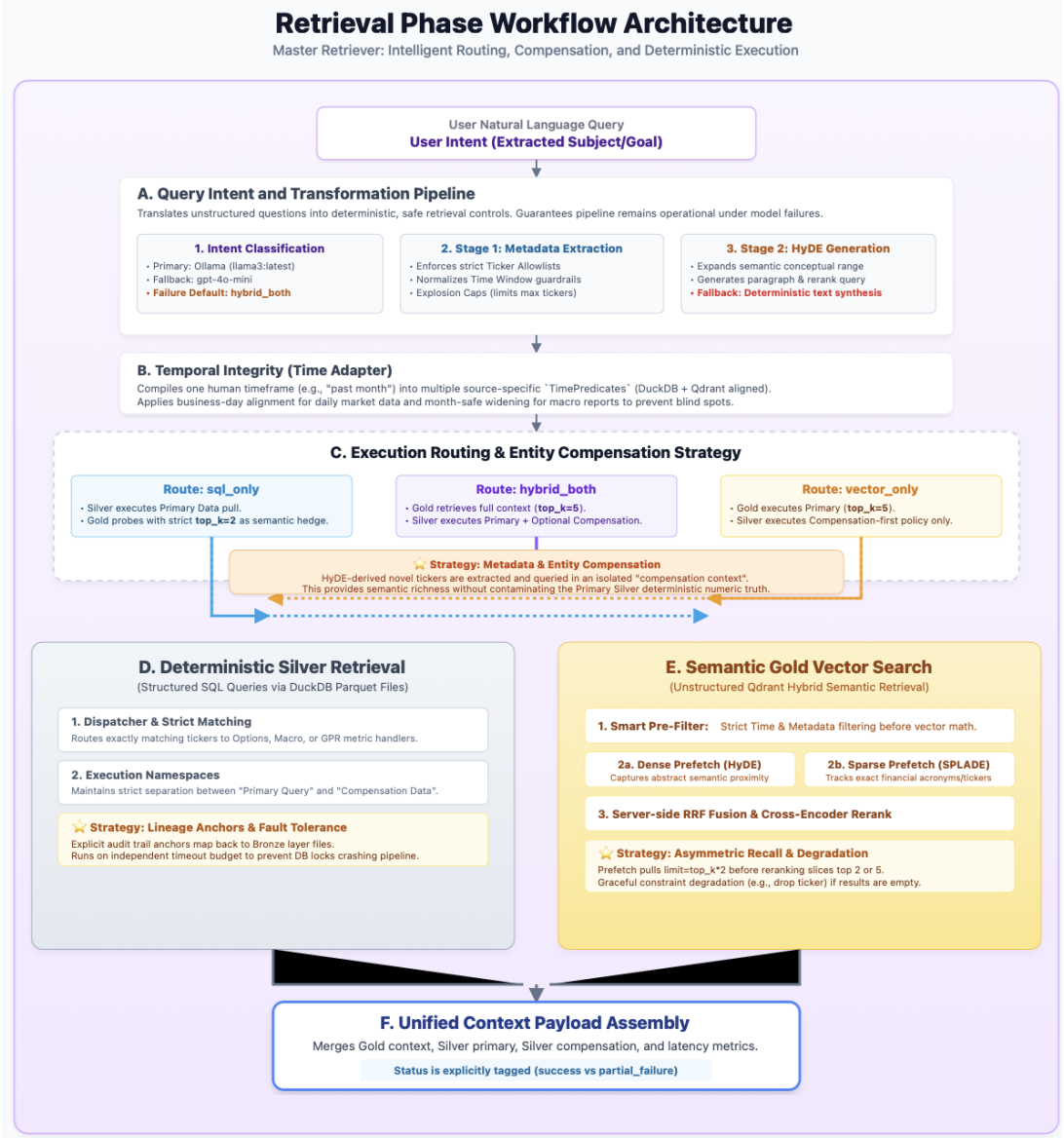
Ingestion Phase



During the ingestion phase, the system operates as a highly controlled pipeline that converts Gold-layer semantic JSONL artifacts into indexed Qdrant points to enable hybrid vector retrieval. To guarantee data integrity, this phase is governed by strict control objectives: it ensures idempotent and replay-safe upsert behavior, guarantees dual-vector completeness, and maintains stable lineage fields for downstream citation and auditing.

The technical workflow begins by bootstrapping the Qdrant collection and reconciling the index schemas to ensure that the database is fully prepared to handle complex payload filtering (such as filtering by ticker, source type, or timestamp). Once the infrastructure is ready, the system loads the JSONL records, parses the text, and normalizes the payload metadata. To prevent duplicate entries during repeated ingestion runs, the system assigns a deterministic ID to each point (such as using a UUIDv5 hash of an SEC accession number). After the identifiers and payloads are assembled, the pipeline simultaneously computes a dense semantic embedding and a sparse lexical embedding for each chunk. Finally, these components are unified into a single Qdrant point structure and batch-upserted into the vector database, with all operations heavily logged for full observability.

Retrieval Phase



A. Query Intent and Transformation

To begin the retrieval phase, the system processes the user's natural language input through a dedicated Query Intent and Transformation module. The primary goal here is to translate unstructured questions into deterministic, safe retrieval controls. Instead of letting the AI arbitrarily decide what to search for, the system routes the query through a heavily guarded transformation pipeline.

First, an Intent Router classifies the request into a specific operational mode: SQL-only for strict numeric queries, vector-only for narrative research, or a hybrid of both. Next, a Query Transformer executes a two-stage process. In the first stage, it extracts structured metadata while applying strict guardrails, such as ticker allowlists, time window validations, and caps on the number of tickers to prevent system overload. In the second stage, it generates a Hypothetical Document Embedding (HyDE) paragraph to conceptually expand the semantic search. If the underlying model ever fails to generate a valid HyDE, the system safely falls back to a synthesized default query, ensuring the pipeline remains operational and continuously writes to the audit trail.

B. Temporal Integrity via Time Adapter

Because financial data operates on vastly different time cadences—such as daily options pricing, monthly macroeconomic reports, and randomly timed SEC filings—applying a single, generic time window across all databases creates massive blind spots. To solve this, we engineered a Temporal Integrity module centered around a custom Time Adapter.

This adapter takes semantic, human-readable timeframes (like "yesterday" or "the past week") and compiles them into source-specific physical predicates that can be executed consistently by both the Qdrant vector store and DuckDB. For example, it automatically applies business-day alignment for daily market data to intelligently skip weekends, employs a month-safe widening policy for macro stats, and enforces minimum lookback windows for rare corporate events. By translating a single user timeframe into multiple, perfectly aligned database rules, the Time Adapter guarantees that all retrieved data respects the chronological reality of its source without mismatching frequencies.

C. Deterministic Silver Retrieval

Once the query is transformed and temporally aligned, the structured data request is handed to the Silver SQL Tooling module. This component serves as our deterministic numeric retrieval engine, ensuring that all mathematical and quantitative outputs are grounded in hard facts to completely eliminate LLM hallucination risks.

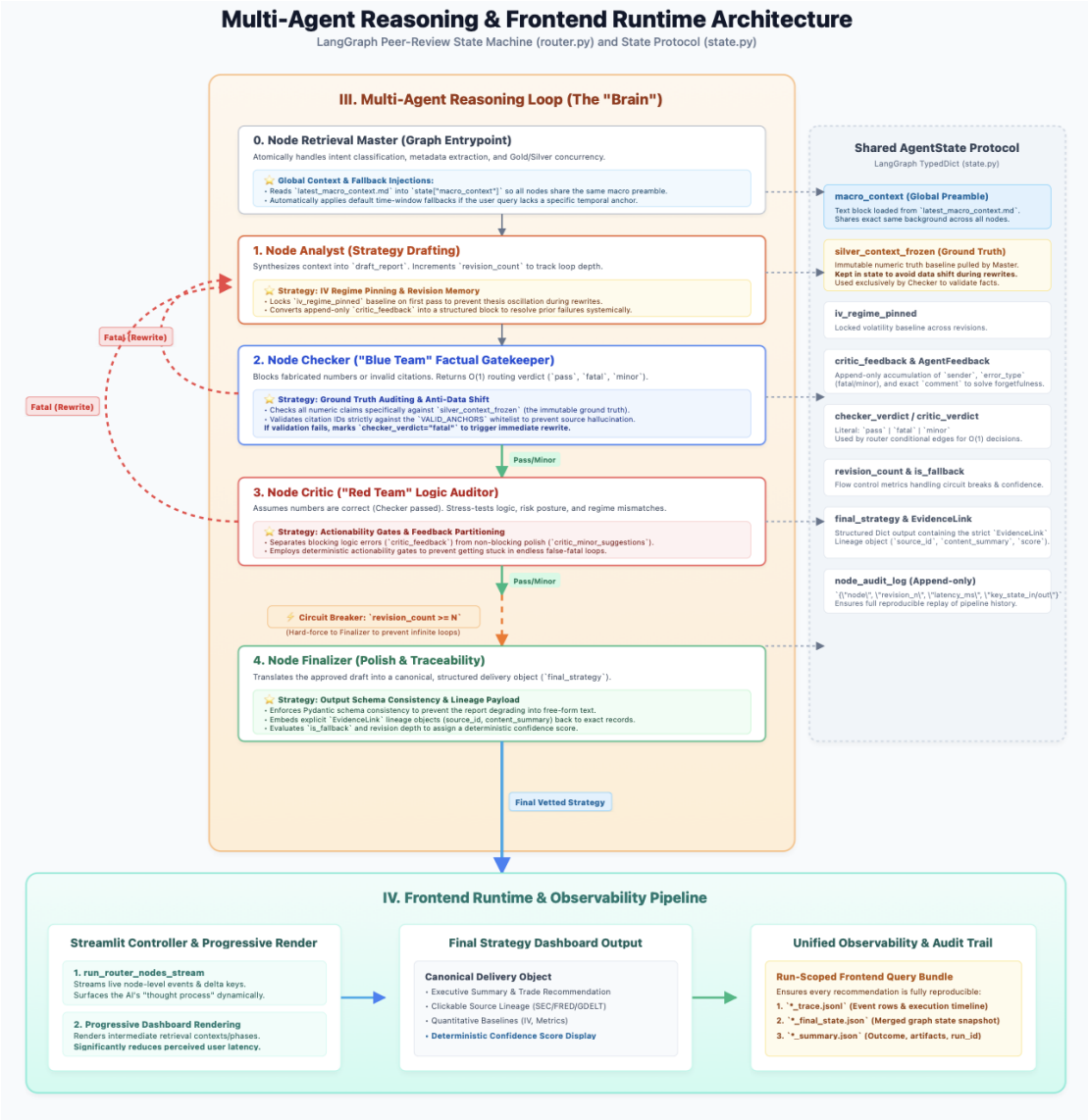
Relying on the extracted metadata, this module strictly enforces whitelist-constrained metric routing and exact ticker matching, meaning it never attempts "fuzzy" guesses on financial symbols. It acts as a central dispatcher, routing the structured query to specialized SQL handlers depending on whether the metric involves options chains, macroeconomics, or geopolitical risks. The module then executes high-performance queries against our Parquet files via DuckDB. Crucially, every successful SQL execution generates explicit lineage anchors and detailed audit logs, providing a fully verifiable trail of the exact quantitative data that gets passed to the final AI reasoning agents.

D. Semantic Gold Vector Search

Operating alongside the deterministic Silver layer, the system executes semantic searches against the Gold-layer Qdrant database to capture unstructured market narratives and complex corporate disclosures. To prevent noisy over-recall, the retrieval engine first applies strict deterministic metadata filters, leveraging the extracted tickers, topics, and source-aligned time predicates, before any vector math occurs.

Once filtered, the engine performs a dual prefetch using asymmetric inputs: it searches the dense vector space using the expanded HyDE paragraph to capture broad conceptual proximity, while simultaneously searching the sparse SPLADE space using short, exact keyword phrases to track specific financial acronyms. Because these models produce incompatible numerical scores, the results are unified server-side using Reciprocal Rank Fusion (RRF). The fused candidates are then passed through a Cross-Encoder reranker, which evaluates the joint relationship between the query and the text to catch critical financial nuances (such as distinguishing between an insider "selling" versus simply "vesting" shares). Finally, if the initial strict filters yield insufficient results, the retriever employs an auditable fallback mechanism that progressively relaxes constraints (e.g., expanding the time window or dropping the ticker requirement), ensuring the downstream multi-agent pipeline always receives the highest-signal context available.

Multi-Agent Workflow



Before the reasoning loop even begins, the system's entry point(the Retrieval Master node) establishes a deterministic data envelope. It gathers all necessary quantitative facts from the Silver layer and semantic narratives from the Gold layer. If the user's query lacks a specific temporal anchor, this node automatically applies default time-window fallbacks to ensure accurate data retrieval. Crucially, the Retrieval Master injects the latest daily global background directly into the agent state. This guarantees that all downstream agents operate on a stable, unified real-world macroeconomic baseline rather than relying on their internal training memory.

Once this data envelope is secured, the payload is passed to the Analyst agent. The Analyst synthesizes the mixed quantitative and qualitative evidence to draft an initial options trading strategy. To prevent the AI's thesis from endlessly oscillating during subsequent rewrites, the Analyst deterministically calculates and locks the implied volatility baseline into the state (iv_regime_pinned) during its first pass. It also increments a revision count to track loop depth and processes any prior append-only feedback to systemically resolve past failures.

After the draft is generated, it moves to the Checker agent, which acts as our "Blue Team" factual gatekeeper. To absolutely prevent "data shift" across multiple rewrites, the Checker does not fetch new data; instead, it audits all numeric claims in the draft strictly against (silver context frozen)—an immutable ground-truth snapshot locked in the shared state. Furthermore, the Checker validates all citation IDs against a strict whitelist (VALID_ANCHORS) to prevent source hallucination. Based on this deterministic audit, it issues an O(1) routing verdict (pass, fatal, or minor). If the Checker detects fabricated numbers or invalid citations, it flags the draft as "fatal" and forces the Analyst to rewrite it.

If the facts are perfectly aligned, the draft passes to the Critic agent. Serving as our "Red Team" logic auditor, the Critic assumes the numbers are correct and focuses entirely on stress-testing the risk posture and overall strategic coherence. It actively looks for logical contradictions, such as proposing a bullish growth strategy during a confirmed high-risk geopolitical regime. To prevent the system from getting stuck in endless false-fatal loops, the Critic employs deterministic actionability gates that strictly separate pipeline-blocking logic errors from non-blocking polish suggestions.

Throughout this peer-review process, a hard circuit breaker constantly monitors the revision_count. If the loop reaches the maximum allowed revisions, the system triggers the circuit breaker and forces the draft directly to the Finalizer to guarantee a timely response. Finally, when the draft survives the Critic's challenge (or triggers the breaker), it reaches the Finalizer agent. The Finalizer translates the approved draft into a canonical, structured JSON delivery object. It embeds explicit, clickable lineage links (Evidence Link) back to the original Bronze-layer SEC filings or news records, and assigns a deterministic confidence score based on the revision depth and pipeline health. The result is a clean, highly polished, and fully traceable options strategy ready for the end user.

VI. Results and Evaluation

AutoOptions successfully demonstrates the viability of a multi-agent Retrieval-Augmented Generation (RAG) architecture in the high-stakes domain of financial research. The deployed system successfully integrates over five active data source families, seamlessly bridging the gap between vastly different time granularities—from daily options pricing to monthly macroeconomic indicators and event-driven corporate filings. By automating the translation of fragmented market evidence into actionable options strategies, the platform effectively closes the information gap for cross-asset financial research.

A primary achievement of this system is its unparalleled reliability. By utilizing deterministic SQL queries against the Parquet tables in the Silver layer, the system anchors the AI in mathematically undeniable facts. This quantitative baseline is then intelligently fused with semantic search results from the Gold layer, ensuring that the AI evaluates market narratives without ever hallucinating the underlying numbers.

Furthermore, the system achieves strict transparency. Every final options recommendation includes explicit, clickable lineage links that route back to the original SEC Form 4/8-K filings, FRED data

sets, or global news articles. This evidence-based formatting ensures a fully traceable decision-making process, allowing human operators to instantly verify the AI's logic before executing a trade.

F. Technical Implementation and Audits

To power our multi-agent architecture, the technical implementation of AutoOptions is divided into a robust backend orchestration layer, a strategically routed LLM pool, an interactive frontend, and a unified observability framework.

Backend Orchestration and Vector Retrieval

The entire backend execution lifecycle is governed by a Python-based CLI, which acts as the canonical runtime surface for data ingestion, model warmup, daemon scheduling, and query serving. To manage our high-selectivity retrieval, we utilize Qdrant as our primary vector store. During execution, the backend maintains strict deterministic control by anchoring run states in a centralized configuration file (collect_data_state.json). This ensures that both the ingestion pipeline and the multi-agent retrieval loops operate on a perfectly synchronized schedule, eliminating any temporal mismatches across our various data sources.

LLM Pool Operations

Because different reasoning tasks require different levels of computational power, we built a centralized LLM Pool module to handle model routing and resource allocation. Our primary domain-tuned reasoning tasks—managed by the Analyst, Checker, Critic, and Finalizer agents—are routed to a locally hosted Llama 3 model via Ollama. Conversely, lighter transformational tasks, such as metadata extraction and HyDE query generation, are routed to an OpenAI-compatible GPT-4o-mini backup instance. To prevent cold-start latency when a user submits a query, the system automatically warms up the heavy agent roles first and leverages an in-process client cache to keep the models actively ready to serve.

Frontend Runtime

For the user interface, we developed a Streamlit-based operator console designed to provide real-time visibility into the system's multi-agent reasoning process. Instead of forcing the user to stare at a loading screen until the entire logic loop finishes, the frontend streams node-level events as they happen. The application progressively renders intermediate retrieval contexts and phase outputs before the full graph completes. This progressive rendering strategy significantly reduces perceived latency and allows the user to watch the AI's "thought process" unfold dynamically before the final structured report is delivered.

Observability and Audit Trails

Finally, we engineered a unified observability framework to ensure that every single recommendation is fully traceable and reproducible. Whenever a query is executed, the orchestrator generates a unique, run-scoped identifier that ties the entire pipeline together. The backend produces partitioned audit logs for retrieval decisions, query transformations, and SQL execution ranges. Simultaneously, the frontend writes a complete query bundle containing event traces, final state snapshots, and summary artifacts. This strict logging discipline guarantees that we always have a

comprehensive, step-by-step JSONL diagnostic trail of exactly how the AI arrived at its final investment recommendation.

VII. Limitations and Further Improvements

Despite its robust architecture, the system operates under several specific limitations dictated by its current data pipeline. Primarily, the platform relies exclusively on end-of-day (EOD) data snapshots; it currently does not support real-time or intraday market querying. If a user asks for live pricing, the system is constrained to its most recent daily anchor. Additionally, the retrieval precision is highly dependent on the quality of the user's prompt. If a user inputs an ambiguous time phrase or fails to include a specific ticker or macroeconomic anchor, the system defaults to a broad six-month lookback window. For fast-moving options markets, this can dilute the retrieval signal and result in overly generalized advice. Finally, queries targeting out-of-universe symbols will encounter a deliberate "Silver evidence gap," causing the system to safely decline the quantitative analysis rather than risk fabricating data.

Future improvements will focus on expanding the data universe to cover a broader range of equities and incorporating intraday data streaming to support real-time volatility monitoring. Ultimately, AutoOptions proves that multi-agent RAG can safely handle the complexities of financial research. By treating deterministic numeric data and unstructured semantic narratives as equally vital components of the reasoning process, we have created a tool that goes far beyond simple text summarization. It successfully acts as a governed, strategic analyst capable of providing a vetted perspective on complex market dynamics.

VIII. Lessons Learned and Iterations

Throughout the development of AutoOptions, we encountered several technical challenges that drove significant architectural iterations. First and foremost, we learned that extreme time sensitivity is one of the most critical factors in automated financial analysis. Because market conditions change rapidly, enforcing strict, source-aware time windows proved absolutely necessary. Without our custom time adapters, the AI risked pulling outdated macroeconomic regimes or stale news to justify current volatility opportunities, which would result in dangerous and inaccurate trading advice.

Furthermore, we quickly realized that numeric integrity is paramount and cannot be entrusted to standard vector databases. Early iterations of our system showed that relying solely on semantic vector search for hard financial facts severely degraded accuracy and led to LLM hallucinations. To solve this, we iterated our design to strictly separate our data pathways. We now handle factual, numeric queries deterministically via high-performance SQL in our Silver tables, while reserving our Gold vector database exclusively for unstructured semantic context and market narratives.

Finally, we learned that building user trust requires highly transparent observability. It is not enough for the AI to simply output a final options strategy; the human operator must be able to quickly verify the logic behind it. To address this, we expanded our logging framework to provide both deep, run-scoped backend traces and an interactive frontend audit dashboard. This final major iteration ensured that every multi-agent revision decision, quantitative calculation, and source lineage is completely transparent and highly auditable for the end user.